



Fixed versus variable time window warehousing strategies in real time

Sergio Gil-Borrás¹ · Eduardo G. Pardo² · Antonio Alonso-Ayuso² · Abraham Duarte²

Received: 8 May 2020 / Accepted: 31 July 2020
© Springer-Verlag GmbH Germany, part of Springer Nature 2020

Abstract

Warehousing includes many different regular activities such as receiving, batching, picking, packaging, and shipping goods. Several authors indicate that the picking operation might consume up to 55% of the total operational costs. In this paper, we deal with a subtask arising within the picking task in a warehouse, when the picking policy follows the order batching strategy (i.e., orders are grouped into batches before being collected) and orders are received online. Particularly, once the batches have been compiled it is necessary to determine the moment in the time when the picker starts collecting each batch. The waiting time of the picker before starting to collect the next available batch is usually known as time window. In this paper, we compare the performance of two different time window strategies: Fixed Time Window and Variable Time Window. Since those strategies cannot be tested in isolation, we have considered: two different batching algorithms (First Come First Served and a Greedy algorithm based on weight); one routing algorithm (S-Shape); and a greedy selection algorithm for choosing the next batch to collect based on the weight.

Keywords Time window · Fixed time window · Variable time window · Online order batching · Warehousing

1 Introduction

The picking operation in a warehouse consist of collecting all the items, demanded by the customers, which are stored in the warehouse. This operation might consume up to 55% of the total operational costs of the warehouse [34]. It is well documented that the picking is highly influenced by the layout

of the warehouse, the storage policy, and the routing strategy followed by the pickers [27]. The picking operation can be divided into two main groups: strict order picking (each order is collected individually) and order batching (orders are grouped into batches before being collected).

In this paper, we focus our attention on the picking policy that follows an order batching strategy. When using this strategy, a family of related optimization problems (order batching problems) emerges. Order batching problems usually look for an efficient organization of the picking operation. The simplest version of this family of problems, usually known as Order Batching Problem (OBP) consist of minimizing the total time needed to collect a group of orders that have been received in a warehouse. This version of the OBP has been tackled by many authors in the literature: Albareda et al. in [1] compared different classical constructive heuristics (First Come First Served, Clarke and Wright [5], and Seed methods [6]) with a Variable Neighborhood Descent [23] metaheuristic over different sets of instances derived from real warehouses. Later, Henn et al. in [14] proposed the use of Iterated Local Search [32] and Ant Colony Optimization [18] to tackle the problem. Again, Henn et al. in [16] introduced the use of other two metaheuristics for the problem: Tabu Search [10] and Attribute-Based Hill Climbing [38]. An exact method based on a mixed-

This research was partially funded by the projects: RTI2018-094269-B-I00 and PGC2018-095322-B-C22 from Ministerio de Ciencia, Innovación y Universidades (Spain); by Comunidad de Madrid and European Regional Development Fund, Grant Ref. P2018/TCS-4566; and by Programa Propio de I+D+i de la Universidad Politécnica de Madrid (Programa 466A).

✉ Eduardo G. Pardo
eduardo.pardo@urjc.es

Sergio Gil-Borrás
sergio.gil@upm.es

Antonio Alonso-Ayuso
antonio.alonso@urjc.es

Abraham Duarte
abraham.duarte@urjc.es

¹ Departamento de Sistemas Informáticos, Universidad Politécnica de Madrid, Madrid, Spain

² Department of Computer Sciences, Universidad Rey Juan Carlos, Móstoles, Spain

integer linear programming model, and a new Iterated Local Search were introduced in [24,25]. More recently, Menéndez et al. in [20,21] proposed a new multi-start method combined with Variable Neighborhood Search which outperformed previous approaches in the state of the art.

Despite of the fact that the simplest version of the OBP is the one with a more extensive literature, other well-known variants of this family of problems have also been tackled in the state of the art. Each variant is characterized by either the optimization of a different objective function or the consideration of different constraints. Among the most outstanding ones, we can find the Min–Max Order Batching Problem, consisting of minimizing the maximum differences among the working-time of a group pickers [12,22,31,39]; or the Order Batching and Sequencing Problem which consists of minimizing the total tardiness of a group of orders with a due date associated to each order [3,13,15,19].

All the previous variants of the order batching family can be considered as static variants, since all the orders are available at the beginning of the process and, therefore, all the information is known beforehand. However, in this paper, we focus our attention on a subtask that occurs within the dynamic version of the OBP, usually known in the literature as Online Order Batching Problem (OOBP). In this variant, only a small part of the orders is available at the beginning of the picking process and, the rest of the orders are received online (i.e., they arrive to the warehouse at any time during a considered time horizon, once the picking process has already begun). In the version of the OOBP considered in this paper, each batch is collected by a single picker, who retrieves all the items belonging to the orders sorted in that batch. Additionally, an order cannot be splitted into different batches. Within this context it is possible to study different objective functions such as: the total time needed to collect all the orders (completion time); the time that pickers spend collecting the orders (picking/service time); the maximum/average time that an order remains in the system (turnover time); the balance of time among the pickers (workload balance); the delays in handling the orders (tardiness); etc.

As in the case of the OBP, the OOBP has also been tackled in the past using heuristic and metaheuristic methods. Particularly, S. Henn in [11] proposed an Iterated Local Search for the OOBP. Later, Pérez et al. in [26] used a method based on the Estimation Distribution Algorithm [17]. Then, Zhang et al. in [41] also introduced several heuristics for the OOBP, but this proposal was not compared with the previous ones. Finally, the most recent approaches have been proposed by Gil et al. in [7,9], where the authors proposed different methods based on the Variable Neighborhood Search metaheuristic, outperforming previous methods in the state of the art. Closely related, Rubrico et al. in [30] studied a multipicker variant of the OOBP by proposing a Steepest Descent Insertion and a Multi-Stage Rescheduling method.

More recently, Gil et al. [8] also handled a multipicker version of the problem by proposing a Basic Variable Neighborhood Search for this variant.

In this paper, we study a subtask which occurs within the context of the OOBP, consisting of determining the time that a picker should wait before start collecting the next available batch. This time is usually known as time window in the literature of the OOBP. The rest of the article is organized as follows: in Sect. 2 we introduce the time window in the context of the OOBP; in Sect. 3 we describe the routing, batching and time window strategies used in the experimental comparison; in Sect. 4 we present the experiments performed; and finally, the conclusions and future works are compiled in Sect. 5.

2 Problem description

In the context of the OOBP it is necessary to study several static and dynamic factors in order to design efficient algorithms to handle the problem. Among the static ones we can find: the warehouse design (number of aisles, width of each aisle, number of blocks, number and position of the depots, etc.), or the distribution of the products in the warehouse (random distribution, ABC distribution, single/multiple locations for the products, etc.), among others. As far as the dynamic factors are concerned, we can find: the number of pickers, the number and size of orders arrived to the warehouse, the due date of some orders, among others.

In this paper, we focus our attention on the OOBP for rectangular-shaped warehouses composed by several parallel aisles and two cross aisles. In Fig. 1 we show an example of the structure of the warehouse considered. In this case, the warehouse is composed by 5 parallel aisles and 2 cross aisles (one at the front and another one at the back of the figure). Each parallel aisle is formed by 9 picking positions at each side of the aisle. The depot (i.e., the point where the routes start and finish) can be placed either at the left most corner or at the centre point of the front cross aisle. In the example warehouse of the Fig. 1, the depot is placed at the centre of the front cross aisle. The details of the set of instances used in our experiments (number of parallel aisles, position of the depot, distribution of the items, etc.) are described in Sect. 4.1.

Additionally, when tackling the OOBP it is necessary to face several subproblems that have been previously handled in the literature: determining how the orders are sorted into batches [7,20]; assigning each batch to a picker [8,11,22]; setting the sequence in which the batches are retrieved [13,19]; and designing the route that the pickers will follow within the warehouse [29,35]. However, not much attention has been paid to determining the time window strategy (i.e., the time that a picker waits before starting to collect the next available

Fig. 1 Warehouse layout
(adapted from [8])

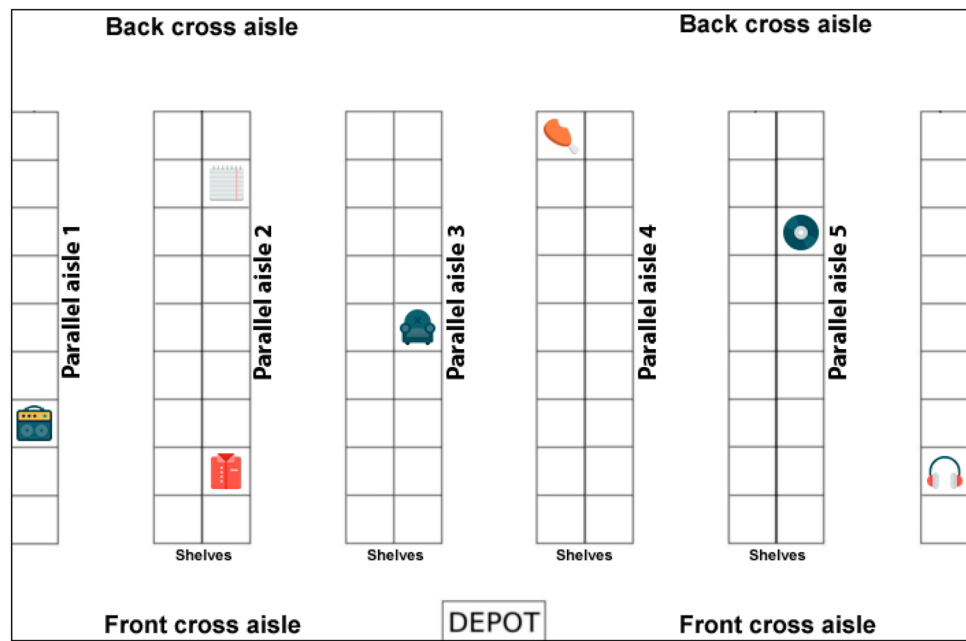
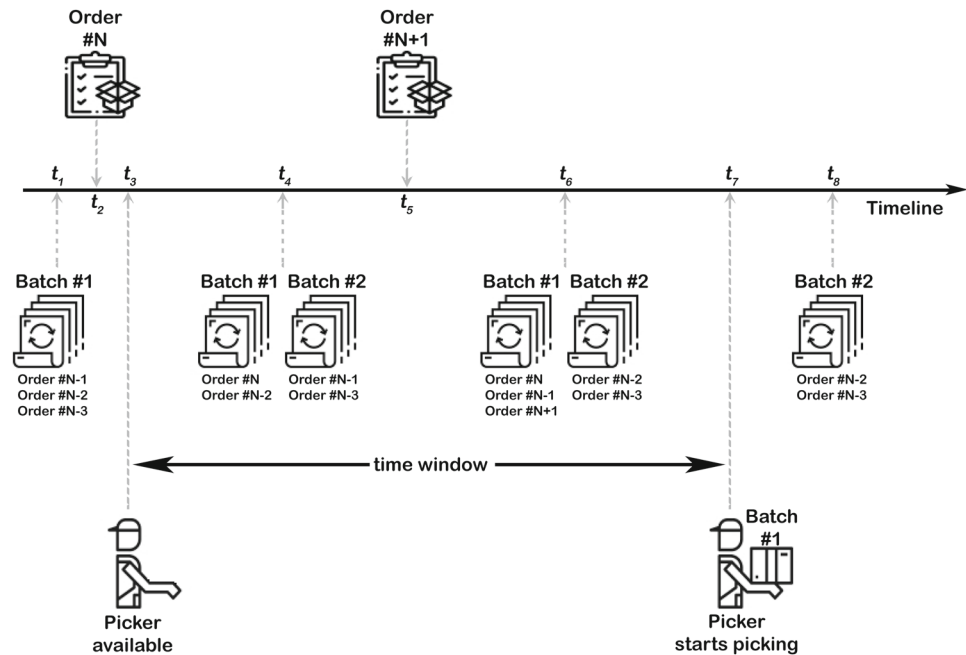


Fig. 2 Events in the timeline of a warehouse



batch). Since we focus our attention on this strategy, in the Fig. 2, we graphically introduce the concept of time window. In this figure we can observe the occurrence of several events at different moments t_i of the timeline considered. Particularly t_2 and t_5 represent moments in the time when a new order arrives to the system. Notice, that orders can arrive to the system at any time with independence of the rest of the tasks being performed in the warehouse (i.e., online arrivals). On the other hand, t_1 , t_4 , t_6 and t_8 represent moments in the time when a new solution has been found by the batching algorithm. Notice that the batching process is continuously

been performed, trying to fit all the orders arrived to the system, not previously collected, into new batches. Finally, t_3 represents the moment in the time when the picker becomes available (i.e., it has already handled all the orders from the previous batch into the depot). Similarly, t_7 represents the moment in the time when a picker starts collecting the next available batch (i.e., Batch #1 in the figure). Given this context, the time window is defined as the difference in the time between the moments t_7 and t_3 .

As we will show in the experimental section of this paper, we can affirm that the strategy used to determine the time

window has also a deep impact in the picking task. Generally speaking, there are two main groups of strategies to handle the time window subtask: Fixed Time Window (FTW) strategy and Variable Time Window (VTW) strategy. The FTW strategy establishes a fixed amount of time that a picker waits before starting to collect the next available batch. On the other hand, in the VTW the time that the picker waits varies through the time. In this sense there are many different VTW strategies. However, the most common one is based on the rate of orders that arrive to the system.

In the context of the OOBP, previous works in the state of the art mention the use of a Time Window strategy to increase the performance of their picking algorithms [4,33,40]. Also, there are references in the literature which use the time window as a part of their batching strategy [28,36]. More recent proposals include new algorithms to determine the time window within a particular context [2,11,36,41]. However, as far as we know, there are no specific comparisons among the time window strategies present in the literature of the OOBP.

This paper is focused on a preliminary comparison between the two main families of time window waiting strategies (FTW and VTW) in the context of the OOBP. With that aim we have considered one well-known routing strategy (S-Shape) and two different batching algorithms. Also, we study the performance of the strategies depending on the objective function reported. In this sense, we consider the picking time (the time that the picker spends collecting items in the warehouse without taking into consideration the waiting time) and the completion time (the total time needed to collect all the orders, including the waiting time).

3 Algorithms

In this section we describe the algorithms used in our experimental comparison. The section is divided considering the subtask of the OOBP tackled. Particularly, the two compared time window strategies are described in Sect. 3.4. However, in order to tackle the OOBP it is also necessary to establish a batching strategy (see Sect. 3.1 for the description of the two batching strategies used), a selection strategy (see Sect. 3.2), and a routing strategy (see Sect. 3.3). The objective is to measure the impact of the time window proposals in combination with the other algorithms for the OOBP.

3.1 Batching algorithm

Here, we present the two batching strategies selected for our experimental comparison. The batching algorithms used can be considered as two basic batching strategies very well-known in the literature of Order Batching problems.

- **First Come First Served (FCFS)** is a very simple batching algorithm which assigns the orders to the batches based on the instant of the time when each order arrives to the system. This is, the first order arriving to the system is assigned to the first available batch, the second order arriving to the system is tried to be assigned to the same batch. If it does not fit in it a new batch is created and the order is assigned to this new batch. The process is repeated by trying to assign orders to the last available batch or creating a new batch if it does not fit in the last available one.
- **Greedy algorithm based on weight** sorts all the orders available in the system (i.e., not collected yet) based on its weight, in the way that the heaviest order come first and the slightest is the last one. Then, it assigns the orders one by one to the batches, by trying first to include the order considered into an already existing batch (with the aim of completing the batch as much as possible) or creating a new batch if the order does not fit into any of the previous ones.

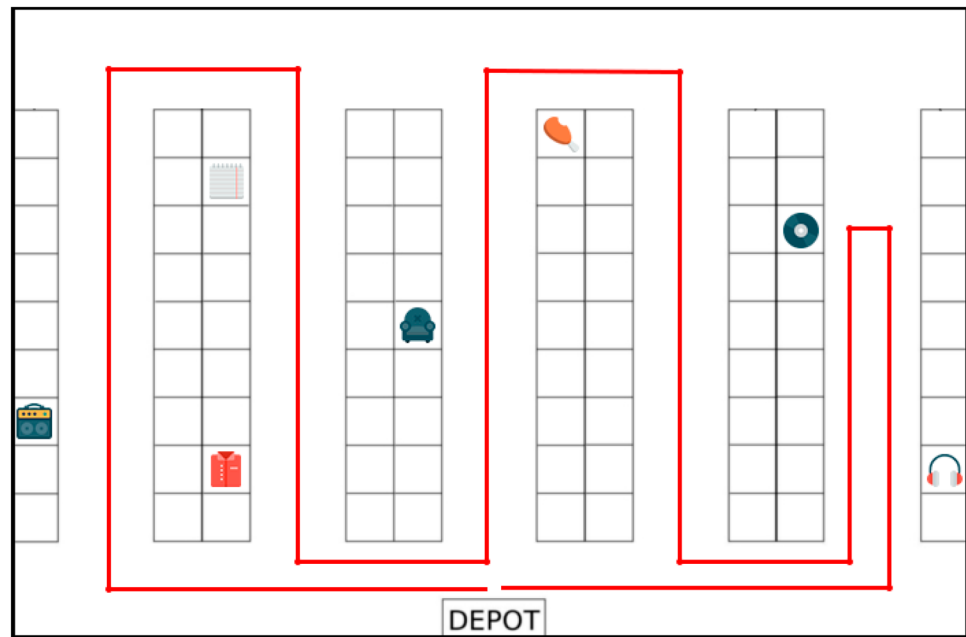
3.2 Selection algorithm

Once the batching algorithm has sorted the orders into batches it is necessary to determine which, among the available batches, will be the next one to be picked. This task might be very important depending on the objective function tackled. For instance, some variants of the OOBP consider a due date for the orders. In these cases, the algorithm which selects the next batch to be collected is a key component of the process. However, in this paper, we do not focus our attention on this task. Therefore, we have used a very simple selection algorithm which is based on the total weight of the orders assigned to each batch. In particular, we calculate, per batch, the sum of the weights of the orders assigned to that batch. Then, we select the heaviest batch as the next batch to collect. If there are more than one batch with the same maximum weight, to break the tie we select the batch which has assigned the shorter path to collect all the orders in that batch. Notice that the length of the path depends on the routing algorithm that is presented in Sect. 3.3.

3.3 Routing algorithm

In the context of the OOBP it is necessary to establish a route for collecting all the items within the orders assigned to the same batch. Notice that all items must be collected in the same route by a single picker. In this paper, we propose the use of a very well-known routing algorithm in the literature (**S-Shape**), in order to determine the path that the picker must follow. All routes start and end at the depot. The S-Shape algorithm establishes that the picker fully traverses each parallel aisle with an item to collect, using the front and

Fig. 3 S-Shape routing strategy (adapted from [8])



back cross aisles to change from one parallel aisle to another one. The first parallel aisle to be traversed will be the leftmost one with items to be collected. If the number of parallel aisles to be traversed is odd, then the picker performs a U-turn in the last parallel aisle in order to return to the front cross aisle, where the depot is placed. In Fig. 3 we show an example of a route designed using S-Shape for the warehouse introduced in Fig. 1.

3.4 Time window algorithm

The time window in the context of the OOBP is the time that a picker is waiting at the depot before starting to collect the next available batch. A naive time window strategy would be a “no-waiting policy”. This policy indicates that the picker starts a new route as soon as he/she finishes the previous one and there are orders awaiting to be collected. However, a more elaborated strategy might result in a better performance. The main contribution of this paper is the comparison of two families of time window strategies together with two objective functions, two batching algorithms, and one routing algorithm. Next, we describe the time window strategies compared:

- **Fixed Time Window (FTW)** strategy consists of defining a fixed number of minutes that the picker must wait before starting a new route. In this paper, we have considered three different time horizons: 3, 6, and 12 min. Notice that if after the considered time horizon there is not at least one order available, the picker waits until a new order comes into the system.

- **Variable Time Window (VTW)** strategy varies the waiting time depending on the environmental conditions of the warehouse. In this case, the VTW strategy used is based on the arrival rate of orders. In particular, the picker waits until there are a minimum number of orders awaiting in the system before starting the picking. We have considered the cases of 4, 8, and 16 orders.

It is important to highlight that the most suitable time window strategy might depend on many different factors such as the routing and batching strategies used, the objective function considered, or the static conditions in the warehouse.

4 Experimental comparison

In this section we present the data used in our experiments and the computational results obtained with the algorithms described in Sect. 3. Since our objective is devoted to compare different time window strategies, for each experiment we report the performance of the considered time window algorithms: FTW, and VTW with several configurations. Particularly, FTW has been configured with 3, 6, and 12 waiting minutes. This means, that the picker waits that number of minutes before starting to collect the next available batch. Similarly, the VTW has been configured with 4, 8 and 16 orders, which means that the picker waits until there are at least that number of orders awaiting to be collected. Additionally, in order to set a reference baseline, we have also reported the performance of the no-waiting strategy (the picking of the next batch starts as soon as the picker becomes available).

Table 1 Comparison of Time Window strategies for the OOBP with picking time objective function, using the FCFS batching algorithm and the S-Shape routing algorithm

Picking time			
Method	Dev. (%)	#Best	Avg. (s)
No-waiting	4.91	1	27,934
FTW 3 Min.	4.91	1	27,934
FTW 6 Min.	4.91	1	27,934
FTW 12 Min.	4.04	1	27,764
VTW 4 orders	0.96	7	27,243
VTW 8 orders	0.33	10	27,104
VTW 16 orders	0.04	14	27,057

The time horizon considered for the arrival of orders has been set to 4 h and the distribution of the arrival of the orders follows a Poisson point process [37]. The Poisson point process uses the 4h as an input parameter to distribute the instants in the time for the arrival of each considered order in the instance, which results in an approximate total time of 4h. Notice, that the picker will only collect the orders scheduled by this distribution. However, the picking process might take longer than the 4h considered as the time horizon.

Experiments are organized in two different sections (depending on the batching strategy followed) and, per section, we include two tables depending on the objective function considered. For all tables we report the deviation to the best solution found in the experiment (Dev. (%)), the number of best solutions found in the experiment (#Best), and the average of the objective function (Avg.(s)) measured in seconds. Additionally, we have included, in each section, a figure which graphically compares the quality of the solutions obtained with respect to each objective function. Finally, we have performed a set of statistical tests in order to determine if the differences found in the results are statistically significant.

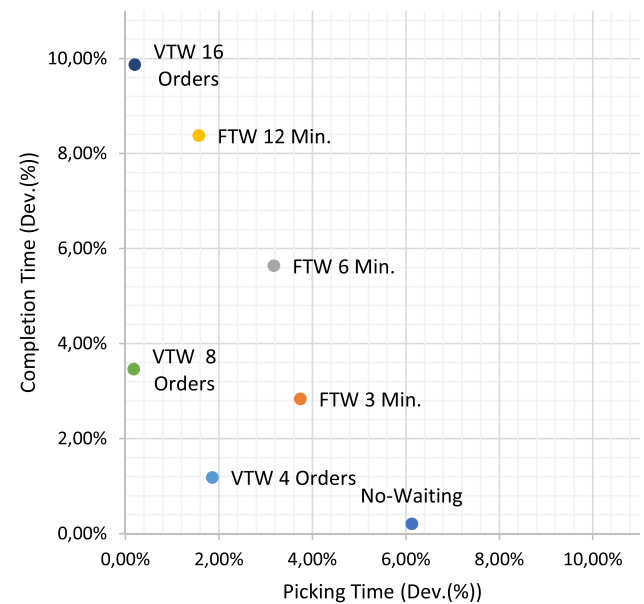
All the experiments were run in an Intel (R) Core (TM) 2 Quad CPU Q6600 2.4 GHz computer, with 4 GB DDR2 RAM memory. The operating system used was Ubuntu 18.04.1 64bit LTS, and all the codes were developed in Java 8.

4.1 Instances

The data set used in our experiments is composed by 16 diverse instances derived from a larger dataset originally proposed in [11]. The size of the selected instances ranges from 40 to 100 orders. The one-block, rectangular-shaped warehouse considered is similar to the one presented in Sect. 2. In this case, all the instances are referred to a warehouse with 10 parallel aisles, and with the depot placed at the centre of the front cross aisle. The products are organized following an ABC distribution.

Table 2 Comparison of Time Window strategies for the OOBP with completion time objective function, using the FCFS batching algorithm and the S-Shape routing algorithm

Completion time			
Method	Dev. (%)	#Best	Avg. (s)
No-waiting	0.53	6	28,194
FTW 3 Min.	0.53	5	28,194
FTW 6 Min.	0.53	4	28,194
FTW 12 Min.	0.23	8	28,127
VTW 4 orders	1.38	3	28,430
VTW 8 orders	4.69	0	29,189
VTW 16 orders	10.93	0	30,499

**Fig. 4** Comparison of the performance in Dev. (%) of the different variants of the algorithms configured with the FCFS batching strategy for the two objective functions**Table 3** Friedman rank test for each objective function when using the FCFS batching method

FCFS			
Picking time		Completion time	
Method	Rank val	Method	Rank val.
VTW 16 orders	1.91	FTW 12 Min.	2.72
VTW 8 orders	2.28	FTW 6 Min.	2.88
VTW 4 orders	2.63	No-waiting	3.03
FTW 12 Min.	4.69	FTW 3 Min.	3.13
FTW 6 Min.	5.50	VTW 4 orders	3.63
FTW 3 Min.	5.50	VTW 8 orders	5.72
No-waiting	5.50	VTW 16 orders	6.91

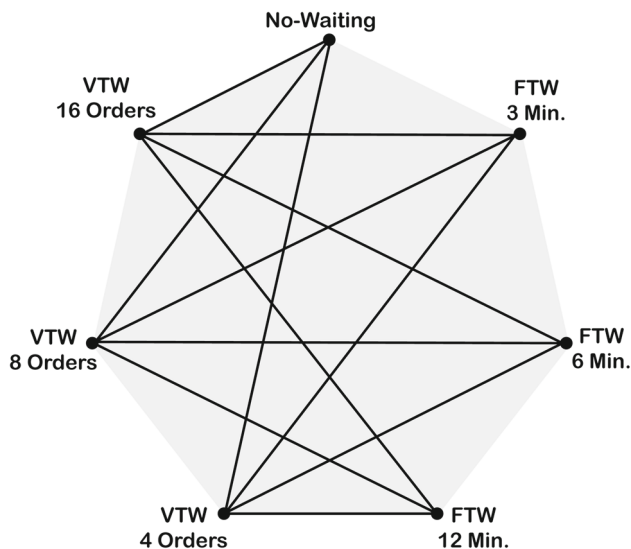


Fig. 5 Representation of the Wilcoxon test comparison by pairs of methods, for the picking time objective function and FCFS batching strategy

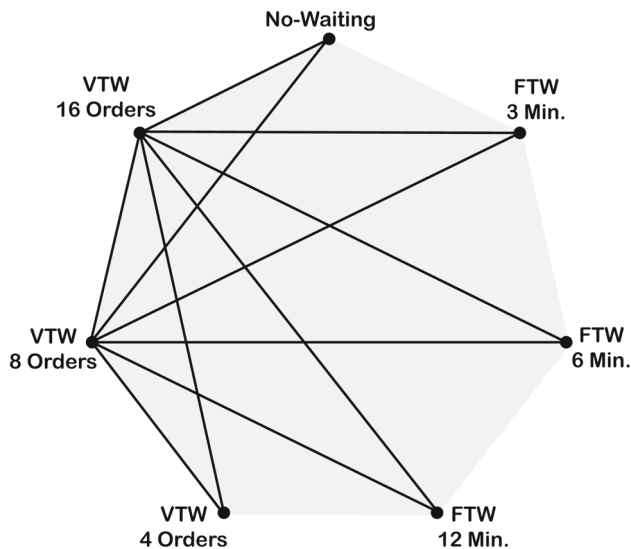


Fig. 6 Representation of the Wilcoxon test comparison by pairs of methods, for the completion time objective function and FCFS batching strategy

4.2 Comparison of time window strategies using FCFS batching algorithm

We compare the performance of the algorithms designed for the OOBP configured with FCFS as the batching method, S-Shape as the routing method and the two Time Window strategies configured with 3 different parameters each. In Tables 1 and 2 we report, respectively, the results obtained for the “picking time” and the “completion time” objective functions. For the instances considered in these experiments, we observe, in Table 1, that all the strategies based on VTW

outperform the results by the FTW ones. Particularly, the best strategy for the picking time objective function is the VTW with 16 orders in terms of deviation to best solution found and in the number of best solutions. On the other hand, in Table 2 we observe the results for the completion time objective function. In this case, all the strategies based on the FTW approach outperform the VTW ones. In particular, the FTW with 12 min is the best strategy closely followed by the no-waiting strategy and the other FTW configurations.

Since the best performance for each objective function has been reached by a different Time Window algorithm, in Fig. 4 we compare the performance of the different configurations of the algorithms with respect to both objective functions. Observing the figure, we find that the performance of some configurations is very good in one of the objective functions and very poor in the other. However, the algorithm which includes VTW with 4 orders finds a balance between the two objective functions compared. Notice that, as it is shown in the figure, there are no differences among the no-waiting strategy, the FTW with 3 min strategy and the FTW with 6 min, when considering both objective functions.

In order to confirm if the differences found among the methods in Tables 1 and 2 are statistically significant, we have conducted a set of statistical tests. First, we have compared all the methods using the Friedman Rank test per table. In both cases, the obtained p values of 0.000 indicate that the methods differ among them, when considering all of them together. Additionally, in Table 3 we report the rank values obtained for each method in both: the picking time and the completion time. As we can observe some of the methods differ greatly, but others achieve the same or almost the same rank value. To complement this test, we have compared each pair of methods individually, with the Wilcoxon signed rank test, in order to identify differences between the objective function values of the best solutions found by each method with respect to the rest of them. In Figs. 5 and 6 we report a graphical representation of the differences by pairs, among the 7 methods compared, for the picking time and the completion time objective functions, respectively. In both figures, a black line between each pair of methods indicates that there are statistically significant differences between them, while the absence of a link between a pair of methods indicates that there are no differences between them.

4.3 Comparison of time window strategies using Greedy batching algorithm

Similarly, in this section, we compare the performance of the algorithms designed for the OOBP configured with the Greedy as the batching method, S-Shape as the routing method and different configurations of the Time Window strategies. In Tables 4 and 5 we respectively report the results obtained for the “picking time” and “completion time”

Table 4 Comparison of Time Window strategies for the OOBP with picking time objective function, using the Greedy batching algorithm and the S-Shape routing algorithm

Picking time			
Method	Dev. (%)	#Best	Avg. (s)
No-waiting	6.13	0	26,407
FTW 3 Min.	3.74	0	26,042
FTW 6 Min.	3.17	0	25,955
FTW 12 Min.	1.57	2	25,675
VTW 4 orders	1.86	2	25,668
VTW 8 orders	0.18	4	25,375
VTW 16 orders	0.20	9	25,383

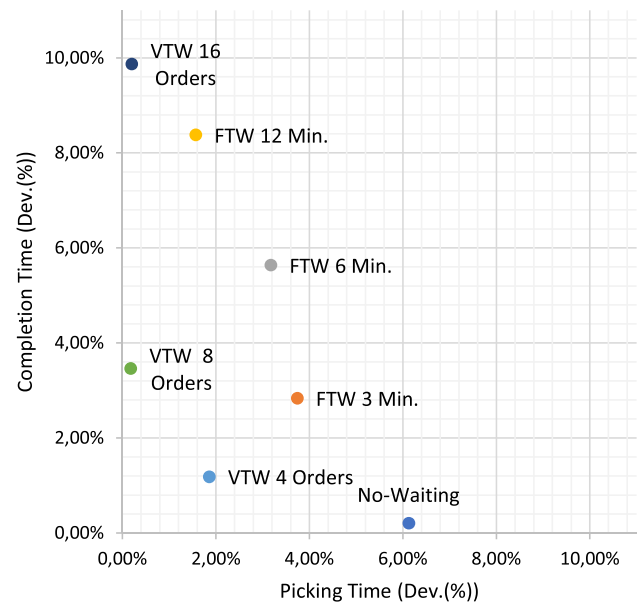
Table 5 Comparison of Time Window strategies for the OOBP with completion time objective function, using the Greedy batching algorithm and the S-Shape routing algorithm

Completion time			
Method	Dev. (%)	#Best	Avg. (s)
No-waiting	0.21	9	26,667
FTW 3 Min.	2.84	0	27,248
FTW 6 Min.	5.64	0	27,832
FTW 12 Min.	8.38	0	28,380
VTW 4 orders	1.19	6	26,892
VTW 8 orders	3.46	1	27,450
VTW 16 orders	9.87	0	28,826

objective functions. For the instances considered in these experiments, when the objective function is the picking time, we observe that the best strategy is the VTW with 8 orders closely followed by the VTW with 16 orders (see Table 4). On the other hand, in Table 5 we observe that, when the objective function is the completion time, the best strategy is the no-waiting one.

Again, in Fig. 7 we compare the performance of the methods over the two objective functions simultaneously. In this case, the VTW with 4 orders and the VTW with 8 orders are the methods which find a better balance between both objective functions.

As in the case of the FCFS batching method, in order to confirm if the differences found among the methods in Tables 4 and 5 are statistically significant, we have conducted a Friedman Rank test per table. In both cases, the obtained p values of 0.000 indicate that the methods differ among them, when considering all of them together. However, in Table 6 we report the rank values obtained for each method in both: the picking time and the completion time for the Greedy batching method. As it was the case of the FCFS batching method, we can observe some of the methods differ greatly, but others achieve the same or almost the same rank

**Fig. 7** Comparison of the performance in Dev.(%) of the different variants of the algorithms configured with the Greedy batching strategy for the two objective functions**Table 6** Friedman rank test for each objective function when using the Greedy batching method

Greedy			
Picking time		Completion time	
Method	Rank val.	Method	Rank val.
VTW 16 orders	2.06	No-waiting	1.50
VTW 8 orders	2.09	VTW 4 orders	2.19
VTW 4 orders	3.38	FTW 3 Min.	3.23
FTW 12 Min.	3.38	VTW 8 orders	3.97
FTW 6 Min.	5.13	FTW 6 Min.	4.75
FTW 3 Min.	5.31	FTW 12 Min.	5.69
No-waiting	6.66	VTW 16 orders	6.66

value. To complement this test, we have also compared each pair of methods individually, with the Wilcoxon signed rank test, in order to identify differences between the objective function values of the best solutions found by each method with respect to the rest of them. In Figs. 8 and 9 we report a graphical representation of the differences by pairs among the 7 methods compared, for the picking time and the completion time objective functions, respectively. In both figures, a black line between each pair of methods indicates that there are statistically significant differences between them, while the absence of a link between a pair of methods indicates that there are no differences between them.

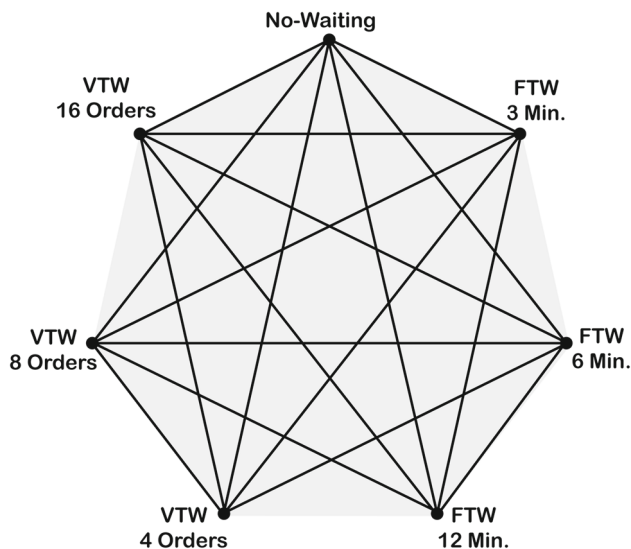


Fig. 8 Representation of the Wilcoxon test comparison by pairs of methods, for the picking time objective function and the Greedy batching strategy

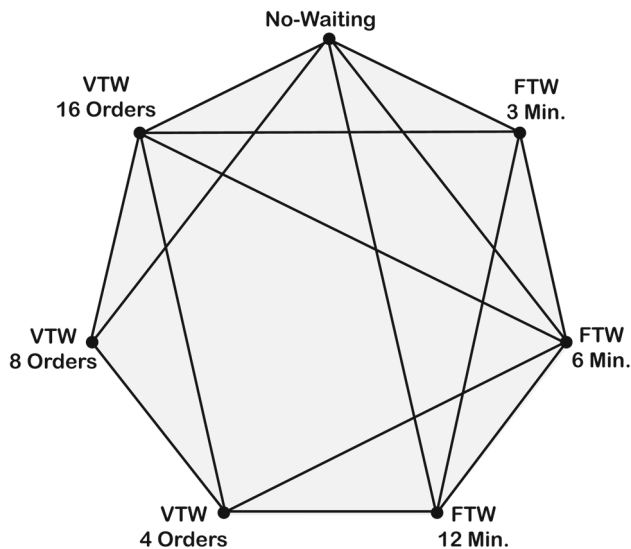


Fig. 9 Representation of the Wilcoxon test comparison by pairs of methods, for the completion time objective function and the Greedy batching strategy

5 Conclusions and future work

In this paper, we have compared two families of time window strategies in the context of the Online Order Batching Problem. The compared strategies have been evaluated together with two different batching algorithms and with two different objective functions. However, we have only considered one routing algorithm (S-Shape) for all the methods compared.

Our main conclusion is that the time window strategy highly impacts in the quality of the solutions obtained in the context of the OOBP. Also, we observed that the most suitable

time window strategy might depend on the objective function evaluated, and on the routing and batching algorithms considered. However, the influence of the time window when using the FCFS batching algorithm seems to be smaller than in the case of using a more elaborated one. Furthermore, when more than one objective function is optimized at the same time, some time window strategies find a balance in the quality of the solution.

Given the previous findings, we suggest to enlarge this study by performing a more elaborated comparison. In particular, the most outstanding time window algorithms in the state of the art should be considered and paired with different batching and routing strategies. Also, several warehouse configurations could be evaluated by enlarging the data set used in our experiments.

References

1. Albareda-Sambola, M., Alonso-Ayuso, A., Molina, E., De Blas, C.S.: Variable neighborhood search for order batching in a warehouse. *Asia Pac. J. Oper. Res.* **26**(5), 655–683 (2009)
2. Bukchin, Y., Khmelnitsky, E., Yakuel, P.: Optimizing a dynamic order-picking process. *Eur. J. Oper. Res.* **219**(2), 335–346 (2012)
3. Cano, J.A., Correa-Espinal, A.A., Gómez-Montoya, R.A.: Mathematical programming modeling for joint order batching, sequencing and picker routing problems in manual order picking systems. *J. King Saud Univ. Eng. Sci.* **32**(3), 219–228 (2020)
4. Chew, E.P., Tang, L.C.: Travel time analysis for general item location assignment in a rectangular warehouse. *Eur. J. Oper. Res.* **112**(3), 582–597 (1999)
5. Clarke, G., Wright, J.W.: Scheduling of vehicles from a central depot to a number of delivery points. *Oper. Res.* **12**(4), 568–581 (1964)
6. Gibson, D.R., Sharp, G.P.: Order batching procedures. *Eur. J. Oper. Res.* **58**(1), 57–67 (1992)
7. Gil-Borrás, S., Pardo, E.G., Alonso-Ayuso, A., Duarte, A.: New VNS variants for the online order batching problem. In: Sifaleras, A., Salhi, S., Brimberg, J. (eds.) *Variable Neighborhood Search. ICVNS 2018. Lecture Notes in Computer Science*, vol. 11328, pp. 89–100. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-15843-9_8
8. Gil-Borrás, S., Pardo, E.G., Alonso-Ayuso, A., Duarte, A.: Basic VNS for a variant of the online order batching problem. In: Benmansour, R., Sifaleras, A., Mladenović, N. (eds.) *Variable Neighborhood Search. ICVNS 2019. Lecture Notes in Computer Science*, vol. 12010, pp. 17–36. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-44932-2_2
9. Gil-Borrás, S., Pardo, E.G., Alonso-Ayuso, A., Duarte, A.: Grasp with variable neighborhood descent for the online order batching problem. *J. Global Optim.* (2020). <https://doi.org/10.1007/s10898-020-00910-2>
10. Glover, F., Laguna, M.: *Tabu Search*. Kluwer Academic Publishers, Norwell (1997)
11. Henn, S.: Algorithms for on-line order batching in an order picking warehouse. *Comput. Oper. Res.* **39**(11), 2549–2563 (2012)
12. Henn, S.: Variable neighborhood search for the order batching and sequencing problem with multiple pickers. Technical report, Otto-von-Guericke University Magdeburg, Faculty of Economics and Management (2012)

13. Henn, S.: Order batching and sequencing for the minimization of the total tardiness in picker-to-part warehouses. *Flex. Serv. Manuf. J.* **27**(1), 86–114 (2015)
14. Henn, S., Koch, S., Doerner, K.F., Strauss, C., Wäscher, G.: Metaheuristics for the order batching problem in manual order picking systems. *Bus. Res.* **3**(1), 82–105 (2010)
15. Henn, S., Schmid, V.: Metaheuristics for order batching and sequencing in manual order picking systems. *Comput. Ind. Eng.* **66**(2), 338–351 (2013)
16. Henn, S., Wäscher, G.: Tabu search heuristics for the order batching problem in manual order picking systems. *Eur. J. Oper. Res.* **222**(3), 484–494 (2012)
17. Larrañaga, P., Lozano, J.A.: Estimation of Distribution Algorithms: A New Tool for Evolutionary Computation, vol. 2. Springer, Berlin (2001)
18. López-Ibáñez, M., Stützle, T., Dorigo, M.: Ant colony optimization: a component-wise overview. In: Martí, R., Panos, P., Resende, M. (eds.) *Handbook of Heuristics*, pp. 371–407. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-07153-4_21-1
19. Menéndez, B., Bustillo, M., Pardo, E.G., Duarte, A.: General variable neighborhood search for the order batching and sequencing problem. *Eur. J. Oper. Res.* **263**(1), 82–93 (2017)
20. Menéndez, B., Pardo, E.G., Alonso-Ayuso, A., Molina, E., Duarte, A.: Variable neighborhood search strategies for the order batching problem. *Comput. Oper. Res.* **78**, 500–512 (2017)
21. Menéndez, B., Pardo, E.G., Duarte, A., Alonso-Ayuso, A., Molina, E.: General variable neighborhood search applied to the picking process in a warehouse. *Electron. Notes Discrete Math.* **47**, 77–84 (2015)
22. Menéndez, B., Pardo, E.G., Sánchez-Oro, J., Duarte, A.: Parallel variable neighborhood search for the min–max order batching problem. *Int. Trans. Oper. Res.* **24**(3), 635–662 (2017)
23. Mladenović, N., Hansen, P.: Variable neighborhood search. *Comput. Oper. Res.* **24**(11), 1097–1100 (1997)
24. Öncan, T.: MILP formulations and an iterated local search algorithm with Tabu thresholding for the order batching problem. *Eur. J. Oper. Res.* **243**(1), 142–155 (2015)
25. Öncan, T., Çağırıcı, M.: MILP formulations for the order batching problem in low-level picker-to-part warehouse systems. *IFAC Proc. Vol.* **46**(9), 471–476 (2013)
26. Pérez-Rodríguez, R., Hernández-Aguirre, A., Jöns, S.: A continuous estimation of distribution algorithm for the online order-batching problem. *Int. J. Adv. Manuf. Technol.* **79**(1), 569–588 (2015)
27. Petersen, C.G.: An evaluation of order picking routeing policies. *Int. J. Oper. Prod. Manag.* **17**(11), 1098–1111 (1997)
28. Quinn, E.B.: Simulation of order processing waves. *Mater. Handling Focus* **83**, 5 (1983)
29. Roodbergen, K.J., Koster, R.D.: Routing methods for warehouses with multiple cross aisles. *Int. J. Prod. Res.* **39**(9), 1865–1883 (2001)
30. Rubrico, J.I.U., Higashi, T., Tamura, H., Ota, J.: Online rescheduling of multiple picking agents for warehouse management. *Robot. Comput. Integr. Manuf.* **27**(1), 62–71 (2011)
31. Scholz, A., Schubert, D., Wäscher, G.: Order picking with multiple pickers and due dates-simultaneous solution of order batching, batch assignment and sequencing, and picker routing problems. *Eur. J. Oper. Res.* **263**(2), 461–478 (2017)
32. Stützle, T., Ruiz, R.: Iterated local search. In: Martí, R., Panos, P., Resende, M. (eds.) *Handbook of Heuristics*, pp. 579–605. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-07153-4_8-1
33. Tang, L.C., Chew, E.P.: Order picking systems: batching and storage assignment strategies. *Comput. Ind. Eng.* **33**(3), 817–820 (1997). Selected Papers from the Proceedings of 1996 ICC&IC
34. Tompkins, J.A., White, J.A., Bozer, Y.A., Tanchoco, J.M.A.: *Facilities Planning*. Wiley, Chichester (2010)
35. Valle, C.A., Beasley, J.E., da Cunha, A.S.: Optimally solving the joint order batching and picker routing problem. *Eur. J. Oper. Res.* **262**(3), 817–834 (2017)
36. Van Nieuwenhuyse, I., de Koster, R.B.M.: Evaluating order throughput time in 2-block warehouses with time window batching. *Int. J. Prod. Econ.* **121**(2), 654–664 (2009)
37. von Bortkiewicz, L.: *Das Gesetz der kleinen Zahlen*. B.G. Teubner, Berlin (1898)
38. Whitley, I.M., Smith, G.D.: The attribute based hill climber. *J. Math. Model. Algorithms* **3**(2), 167–178 (2004)
39. Yokota, T.: Min-max-strategy-based optimum co-operative picking with AGVS in warehouse. In: 2019 58th Annual Conference of the Society of Instrument and Control Engineers of Japan (SICE), pp. 236–242. IEEE (2019)
40. Yu, M., De Koster, R.B.M.: The impact of order batching and picking area zoning on order picking system performance. *Eur. J. Oper. Res.* **198**(2), 480–490 (2009)
41. Zhang, J., Wang, X., Chan, F.T.S., Ruan, J.: On-line order batching and sequencing problem with multiple pickers: a hybrid rule-based algorithm. *Appl. Math. Model.* **45**(Supplement C), 271–284 (2017)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.